COMPILER DESIGN LABORATORY · PHASE 3 REVIEW

CodeFlow

Phase 3 — Optimization, Execution & Integration

Constant Folding · Unreachable & Dead-Code Elimination · TAC Execution Engine · Compiler Explorer

Daksh Arora · B.Tech Computer Science and Engineering (Information Security) · VIT Vellore
Register Number: 24BCI0259 · Course Code: BCSE307P · Individual Project
GitHub: https://github.com/dakshcodez/CodeFlow_DSL

OVERVIEW

Phase 3 at a Glance

Phases 1 and 2 built the front and middle end. Phase 3 closes the loop: TAC is now optimized and then actually executed, so a CodeFlow program runs from source to an alert / log / action trace.

3	131	4	6/6
optimization passes	**automated tests passing**	**runtime error kinds**	**pipeline stages complete**
Folding · unreachable · dead code	10 Vitest suites, 50 new since Phase 2	Missing input · type · divide-by-0 · operand	Source through execution trace

Phase 3 target Source → Tokens → AST → Semantic → TAC → Optimized TAC → Execution → Trace

Complete Pipeline After Phase 3

Every stage is implemented, consumes the previous stage's artifact, and is inspectable.

ERROR HANDLER — Lexical · Syntax · Semantic · Runtime diagnostics, one shared CompilerError

1	2	3	4	5	6
Lexical	**Syntax**	**Semantic**	**IR Gen**	**Optimizer**	**Execution**
Tokens with line/col	Recursive descent → AST	Scope + types, symbol table	Structured TAC	Fold · unreachable · dead	Interpret TAC → trace
COMPLETE	COMPLETE	COMPLETE	COMPLETE	NEW · PHASE 3	NEW · PHASE 3

COMPILER EXPLORER — Tokens · AST · Symbol Table · Errors · TAC · Optimized TAC · Execution Trace

Phase 3 adds stages 5 and 6 and turns the Phase 2 "Phase 3" placeholders (optimized TAC, trace, runtime diagnostics) into live features.

Optimizer: Three Passes, One Order

Design

- Operates on IRInstruction[] only — never sees source text or the AST.
- Each pass is an exported, independently testable function.
- optimizeWorkflow composes them: fold → unreachable → dead.
- Order matters: folding exposes constant conditions, unreachable removal uses them, dead-code removal sweeps what is left unread.
- Out of scope by design: register allocation, scheduling, inter-procedural work.

Pass ordering on a program

1. **`foldConstants`**
   10 + 20 → 30 ; 5 > 3 → true
2. **`eliminateUnreachableCode`**
   drop bodies behind a known-false jump
3. **`eliminateDeadCode`**
   drop temporaries nobody reads

Constant Folding

Before

```
t2 = 10
t3 = 20
t4 = t2 + t3
t5 = t1 > t4
```



After folding

```
t2 = 10
t3 = 20
t4 = 30
t5 = t1 > t4
```

The now-unused t2 and t3 are removed later by dead-code elimination.

How it works

- Tracks the known constant value of each temporary.
- Binary / Unary with constant operands become one Assign.
- Knowledge resets at every Label — a control-flow merge point.

Safety rules

- Sensor / input values are never constants.
- x / 0 is left unfolded so the runtime reports it.
- Mixed-type operands are left alone.

Unreachable-Code Elimination

Algorithm

- Build a small control-flow graph over instruction indices: fall-through edge plus a jump edge for Goto / IfFalse.
- Breadth-first search from instruction 0; keep only reachable instructions.
- If an IfFalse condition is a known boolean, follow only the edge it can take.
- This is why a statically false when loses its body.

Example: when 2 > 9 { action never_called() }

```
t11 = false
IF_FALSE t11 GOTO L3
CALL never_called()
LABEL L3
```



```
t11 = false
IF_FALSE t11 GOTO L3
LABEL L3
```

The CALL is unreachable and is removed.

Dead-Code Elimination

Rule

- Remove an Assign / Binary / Unary whose result temporary is never read by a later instruction.
- Calls, alerts, logs and jumps are never removed — they are the program's observable effects.
- One backward pass is enough: CodeFlow has no loops, so every jump target lies after its jump and each temporary has one static definition.

Real output after folding

```
t1 = temperature
t2 = 10      ; never read
t3 = 20      ; never read
t4 = 30
t5 = t1 > t4
```

↓

```
t1 = temperature
t4 = 30
t5 = t1 > t4
```

Before / After IR: dead-code.cflow

Generated TAC — 20 instructions

```
t1 = temperature
t2 = 10
t3 = 20
t4 = t2 + t3
t5 = t1 > t4
IF_FALSE t5 GOTO L1
ALERT "High temperature"
LABEL L1
t6 = 5
t7 = 3
t8 = t6 > t7
IF_FALSE t8 GOTO L2
LOG "Always runs"
LABEL L2
t9 = 2
t10 = 9
t11 = t9 > t10
IF_FALSE t11 GOTO L3
CALL never_called()
LABEL L3
```

Optimized TAC — 12 instructions

```
t1 = temperature
t4 = 30
t5 = t1 > t4
IF_FALSE t5 GOTO L1
ALERT "High temperature"
LABEL L1
t8 = true
IF_FALSE t8 GOTO L2
LOG "Always runs"
LABEL L2
t11 = false
IF_FALSE t11 GOTO L3
LABEL L3
```

temperature = 40 → ALERT: High temperature, LOG: Always runs — identical to the unoptimized program.

Known limit: a jump on a known constant is kept, not collapsed.

Execution Engine

Design

- Interprets optimized TAC — never reparses source.
- State: program counter, map of temporaries, label index.
- Sensor / input values come from a JSON record supplied at run time.
- Actions are observable effects, not real side effects.
- Result: { workflowName, trace, errors } per workflow.

Instruction semantics

Instruction	Runtime behaviour
`Assign / Binary / Unary`	evaluate, store in temporary
`IfFalse / Goto`	set the program counter from the label index
`Label`	no-op target
`Call`	emit Action effect with arguments
`Alert / Log`	emit Alert / Log effect

DEMO

NEW · PHASE 3

Execution in Action

CoolingSystem

Inputs	Execution trace
`{"temperature": 38}`	`ALERT: High temperature` `ACTION: start_fan()`
`{"temperature": 20}`	`LOG: Temperature normal`

OrderProcessing

Inputs	Execution trace

`{"orderAmount": 12000}`	`ACTION: fraud_check()`
	`ACTION: manager_approval()`
	`ACTION: process_payment()`

Both canonical programs from the Phase 1 proposal now run end to end and are kept as regression tests. The branch taken depends only on the supplied runtime values.

Runtime Diagnostics

Behaviour

- First runtime error halts the workflow.
- Reported as a "runtime"-stage CompilerError with line and column.
- The trace produced before the error is kept.
- Detected: missing input, declared-vs-supplied type mismatch, division by zero, operand type mismatch.

Real diagnostics

a / b > 2 with {"a":4,"b":0}

```
Runtime error at line 4, column 10:
Division by zero
```

temperature = "hot"

```
Runtime error at line 4, column 10:
Type mismatch for 'temperature': expected number but
received string ("hot")
```

Pipeline API and CLI

compiler/pipeline.ts

```
compile(source)
  lex → parse → analyze
  → generateIR → optimize
  → { tokens, program, symbolTables,
      ir, optimizedIR, errors }

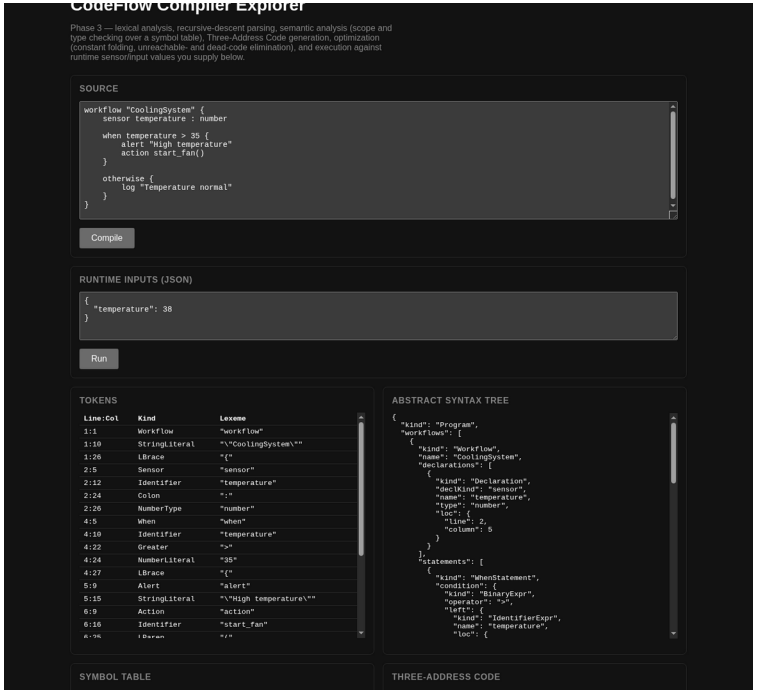
run(source, inputs)
  compile(), then execute(
    optimizedIR, inputs, symbolTables)
```

Why two functions

- Compiling never needs runtime values; running does.
- Every stage runs even if an earlier one reported errors, so all artifacts stay inspectable.
- The CLI prints every artifact; a JSON argument also executes the program.

```
npm run compile -- examples/cooling.cflow
  '{"temperature": 38}'
```

Compiler Explorer — Final

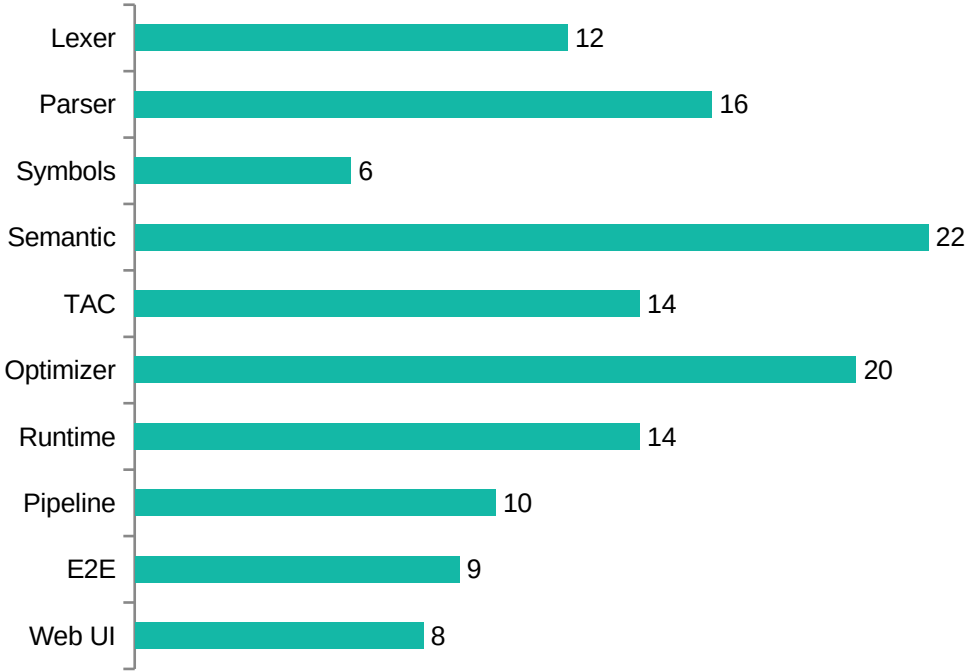


Panels (all live)

- Source and Runtime Inputs (JSON) editors.
- Tokens, AST, Symbol Table.
- Three-Address Code and Optimized TAC side by side for before/after comparison.
- Errors from every stage, with line and column.
- Execution Trace after pressing Run.

Lightweight HTML/CSS/TypeScript bundled with esbuild — no framework, no backend.

Testing by Compiler Stage



131 / 131

tests passing · 10 Vitest suites

- Optimizer: each pass as input IR → output IR, plus unchanged execution semantics.
- Runtime: both branches, dispatch, alerts, logs, runtime errors.
- E2E: source → trace for valid and malformed programs.

PROGRESS

From Phase 2 to Phase 3

Component	After Phase 2	After Phase 3
Lexer / Parser / AST	Complete	Unchanged — regression-tested
Symbol table / Semantic	Complete	Unchanged — feeds the runtime type check
TAC generation	Complete — 9 instruction kinds	Unchanged — now the optimizer's input
Optimizer	Planned	Complete — fold, unreachable, dead code
Execution engine	Planned	Complete — TAC interpreter, runtime errors
Error handling	Lexical · syntax · semantic	+ runtime, same CompilerError shape
Visualization	Tokens · AST · Symbols · TAC · Errors	+ Optimized TAC · inputs · execution trace
Tests	81	131

HONEST NOTES

Limitations and Future Work

Current limitations

- A conditional jump on a known constant (t8 = true; IF_FALSE t8 GOTO L2) is kept, not collapsed.
- Constant knowledge resets at labels; no propagation across merge points.
- Actions are simulated effects only, by design.
- Each workflow runs independently against one shared input record.

Possible next steps

- Rewrite constant-condition IfFalse into Goto or nothing.
- Link each IR line and trace entry back to its source line in the UI.
- Step-through view showing temporaries after each instruction.

ROADMAP

Phase-wise Roadmap

PHASE 1

Problem Definition & System Design

Language specification, formal grammar, architecture, technology stack, lexer and basic parser prototype.

COMPLETE

PHASE 2

Core Compiler Construction

Complete lexer and parser, full AST, symbol table, semantic analysis, TAC generation, layered errors.

COMPLETE

PHASE 3

Optimization, Execution & Integration

Constant folding, unreachable and dead-code elimination, execution engine, before/after IR, full Explorer, E2E tests.

COMPLETE

IN SUMMARY

CodeFlow now compiles, optimizes and runs real programs from source to an execution trace, with every stage inspectable.

- Constant folding, unreachable- and dead-code elimination, shown as before/after IR.
- Execution engine runs the optimized TAC against supplied sensor / input values.
- Runtime errors share the one CompilerError channel with line and column.
- Compiler Explorer shows every artifact; 131 tests pass across all stages.

Thank You

GitHub: https://github.com/dakshcodez/CodeFlow_DSL

Daksh Arora · 24BCI0259 · VIT Vellore